



Cookies: The Invisible Web Helpers

Understanding the Role and Functionality of Web Cookies

Cookies

What Are Cookies?

Cookies are small pieces of text data that a website stores in your browser. Each time your browser sends a request to the same website, it automatically sends the relevant cookies along with the request. This allows websites to remember users, login sessions, preferences, and other information, acting like a small identity card your browser carries for a website.

How Cookies Work

When you visit a website, the browser sends a request to the server. The server responds with a special HTTP header that creates a cookie, which the browser then stores. On subsequent requests to the same website, the browser sends this cookie back to the server, allowing the server to identify the user.

Example:

1. **User Opens Website:** The browser sends an HTTP request to the server.
2. **Server Creates a Cookie:** The server responds with a Set-Cookie header.
3. **Browser Stores Cookie:** The browser saves the cookie.
4. **Next Request:** The browser includes the cookie in future requests to the server.

Real-Life Example

Imagine entering a shopping mall and receiving a visitor card from the security guard. Each time you enter another section, you show this card. Similarly, a cookie acts as your visitor card on a website.

Why Do Websites Use Cookies?

Cookies are used for:

- Login sessions
 - Authentication
 - User preferences
 - Language selection
 - Theme (Dark/Light)
 - Shopping cart
 - Analytics
 - "Remember Me" functionality
-

Creating Cookies with JavaScript

Basic Example:

To create a cookie using JavaScript:

```
document.cookie = "name=Livesh";
```

Reading Cookies:

```
console.log(document.cookie); // Output: name=Livesh
```

Multiple Cookies

You can store multiple cookies in the browser:

```
document.cookie = "name=Livesh";  
document.cookie = "theme=dark";  
document.cookie = "language=en";
```

Output:

```
name=Livesh; theme=dark; language=en
```

Cookie Syntax

```
name=value;  
Expires=date;  
Max-Age=seconds;  
Path=/;  
Domain=example.com;  
Secure;  
HttpOnly;  
SameSite=Lax
```

Example:

```
document.cookie = "theme=dark; path=/";
```

Cookie Properties

1. Expires

Determines when the cookie expires. Without an expiry date, the cookie becomes a session cookie and is removed when the browser closes.

```
document.cookie = "theme=dark; expires=Tue, 01 Jan 2030 12:00:00 UTC";
```

2. Max-Age

Specifies the cookie lifetime in seconds:

```
document.cookie = "theme=dark; max-age=3600"; // 1 hour
```

3. Path

Specifies which URLs can access the cookie. For example, Path=/admin limits access to:

```
/admin  
/admin/dashboard  
/admin/users
```

4. Domain

Specifies which domain can access the cookie:

```
Domain=example.com
```

5. Secure

Ensures the cookie is only sent over HTTPS, not HTTP.

6. HttpOnly

Prevents JavaScript from accessing the cookie, enhancing security.

7. SameSite

Protects against Cross-Site Request Forgery (CSRF). Options include Strict, Lax, and None.

Session Cookies

Session cookies have no expiration date and are deleted when the browser closes.

```
document.cookie = "user=Livesh";
```

Persistent Cookies

These remain until their set expiration date:

```
document.cookie = "user=Livesh; max-age=31536000"; // 1 year
```

Reading Cookies

To read stored cookies:

```
console.log(document.cookie); // Output: name=Livesh; theme=dark; language=en
```

Updating Cookies

To update a cookie, assign the same cookie name again:

```
document.cookie = "theme=light";
```

Deleting Cookies

Set the expiration date to the past:

```
document.cookie = "name=Livesh; expires=Thu, 01 Jan 1970 00:00:00 UTC";
```

Cookies vs Local Storage

Feature	Cookies	Local Storage
Capacity	~4 KB	5–10 MB
Sent to Server	Yes	No
Expiration	Yes	No (until removed)
JavaScript Access	Yes (unless HttpOnly)	Yes
Authentication	Excellent	Not Recommended
Large Data	No	Better

Cookies vs Session Storage

Feature	Cookies	Session Storage
Shared Across Tabs	Yes	No
Sent to Server	Yes	No
Capacity	~4 KB	5–10 MB
Removed on Tab Close	Optional	Yes

Cookies vs IndexedDB

Feature	Cookies	IndexedDB
Storage Size	Very Small	Very Large
Images	No	Yes
Objects	Limited	Yes
Offline Database	No	Yes
Server Communication	Automatic	No

Why Not Store JWT in Local Storage?

Storing JWT tokens in Local Storage can be risky due to potential XSS vulnerabilities. Malicious JavaScript could access and steal the token.

Example:

```
const token = localStorage.getItem("token");
```

Better Approach

Store authentication tokens in a **Secure, HttpOnly Cookie**. This prevents JavaScript from accessing the token and helps protect against XSS token theft. Additionally, use **SameSite** cookies and **CSRF tokens** for enhanced security.

Authentication Flow Using Cookies

1. **User Login:** Browser sends email/password to server.
 2. **Server Verifies User:** Server creates session/JWT.
 3. **Server Sends Set-Cookie Header:** Includes session ID with HttpOnly, Secure, and SameSite=Lax.
 4. **Browser Stores Cookie:** Future requests automatically include the cookie, allowing server identification.
-

Which Storage Should You Use?

Situation	Best Choice
Theme Preference	Local Storage
Shopping Cart	Local Storage (or Backend Sync)
Login Authentication	Secure HttpOnly Cookie
Multi-Step Form	Session Storage
Recently Viewed Products	Local Storage
Offline Notes	IndexedDB
Offline Images	IndexedDB
Large API Cache	IndexedDB
User Preferences	Local Storage
Temporary OTP Process	Session Storage

Summary

- **Cookies** store small pieces of data that are automatically sent with HTTP requests, primarily used for authentication, sessions, and preferences.
- Cookies support security features like **Secure**, **HttpOnly**, and **SameSite**.
- Avoid storing sensitive authentication tokens in Local Storage.
- **Secure HttpOnly Cookies** are often the safest choice for authentication, while **Local Storage**, **Session Storage**, and **IndexedDB** serve different purposes based on the use case.